Firestore Structure — Tree Overview

Every collection and document created under one user. NoSQL document store — collections → documents → subcollections, no fixed schema, no joins.

profile · logs → meals · nbTraining · weightHistory

COLLECTION
a growing folder of documents — has no field count or "1 per X" of its own, it just accumulates

DOCUMENT
one concrete record with actual fields — the "1 per day / 1 per meal" cardinality always describes THIS, never the collection above it

users — COLLECTION
root collection · one document inside it per person who signs up

└ **{uid}** — DOCUMENT
created by **Firebase Auth** at signup · carries no fields of its own — it exists only to hold the 4 subcollections below

├ **profile** — COLLECTION
under this user · will **only ever hold 1 document**, for the account's whole life

│ └ **data** — DOCUMENT
→ **exactly 1, ever.** Created at end of onboarding, then mutated in place forever — never a 2nd doc

├ **logs** — COLLECTION
under this user · gains a new document for every calendar day that has ≥1 meal logged

│ └ **{dateKey}** — DOCUMENT
→ **1 per day.** Implicit — never written directly, starts existing the moment its "meals" subcollection gets a first entry

│ └ **meals** — COLLECTION
under one day · gains a new document every time a meal is logged

│ └ **{autoId}** — DOCUMENT
→ **1 per logged meal.** Created every time the user saves a food entry

├ **nbTraining** — COLLECTION
under this user · gains a new document every calendar day — **this is the "collection", it has no cardinality itself**

│ └ **{dateKey}** — DOCUMENT
→ **1 per day.** This is what "1 per day" actually describes — the document, not the collection above it. Written on dashboard focus (NB feature + label row)

└ **weightHistory** — COLLECTION
under this user · gains a new document every time a weight is logged

└ **{autoId}** — DOCUMENT
→ **1 per check-in.** Created every time the user logs a new weight

USER JOURNEY — WHAT GETS CREATED, IN ORDER

1	2	3	4	5
Sign up	**Onboarding done**	**First meal logged**	**Dashboard opened**	**Weekly check-in**
Firebase Auth account created.	Plan calculated & confirmed.	User logs food that day.	NB pipeline runs on focus.	Weight logged, plan recalcs.
no Firestore doc yet	`profile/data`	`logs/{date}/meals/{id}`	`nbTraining/{date}`	`weightHistory/{id}`

Direct answer: `nbTraining` is the **collection** — it's just a growing folder, it doesn't have a size. The thing that is "1 per day" is the **`{dateKey}` document** sitting inside it. Every teal **COLLECTION** pill above only ever tells you how it grows; every amber **DOCUMENT** pill is where an actual "1 per X" count applies. `profile` is the one collection that stays capped at exactly 1 document forever — every other collection keeps adding documents for as long as the account is used. Pages 2–3 list every field each document type can hold.

Field Reference — profile/data

One document, mutated by nearly every screen. Base fields exist from onboarding onward; the four mode objects are nested maps inside this **same** document — not separate documents — and only appear once that mode has been entered at least once.

P profile / data — BASE FIELDS

name	display name, set at onboarding	**activitySettings**	map: activity name → {daysPerWeek, hoursPerDay}
dob	ISO date string	**bmr**	Mifflin-St Jeor result
age	calculated from dob	**tdee / sedentaryTDEE**	bmr × 1.2
gender	'Male' / other — drives BMR formula branch	**dailyCalorieTarget**	the number shown to the user
height	cm	**foodDeficit**	cut remaining after workout offset
weight	original starting weight, immutable	**dailyDeficit / totalDailyDeficit**	total prescribed deficit
currentWeight	latest logged weight SET LATER	**weeklyWorkoutCalories**	from selected activities
goalWeight	target weight, kg	**onboardingComplete**	bool — routes past onboarding
targetWeeks	plan duration	**createdAt**	ISO timestamp, account creation
targetEndDate	ISO date string	**proteinStreak**	object — see below

↳ proteinStreak — *gamification — always present, defaults to zeros*

currentStreak	consecutive days	lastActiveDate	last date streak advanced
lastCompletedDate	last date goal actually met	shields	earned every 5th completion
totalCompletions	lifetime count	badgeScore	freezes during LM / Injury

↳ activeInjury — *only once Injury Mode has been entered*

active, conditionKey, conditionLabel	which condition	severity, warmMessage, healingNotes	display copy
activitySettings	recovery-safe activities picked	baseBmr, injuryBmr, tdee	condition-adjusted metabolism
recommendedCal	floored at 25 kcal/kg	proteinPct/carbsPct/fatPct	per-condition macro split
proteinG/carbsG/fatG	grams, from recommendedCal	weeklyBurn, dailyBurnAvg	from picked activities
startedAt	ISO timestamp		

↳ loadManagement — *only once LM has been entered*

active, startedAt	—	currentWeight, bmr, sedentaryTDEE	no boost, plain ×1.2
dailyCalories	= sedentaryTDEE, zero deficit	carbsPct/proteinPct/fatPct	fixed 55/20/25
carbsG/proteinG/fatG	—	originalGoalWeight/TargetWeeks/TargetEndDate	snapshot for exit
originalDailyDeficit, originalStartWeight	snapshot for exit	originalActivitySettings	restored on exit

↳ deficitCycling — *only once cycling has been entered*

active, startedAt, currentWeight, goalWeight	—	bmr, sedentaryTDEE, tdee	tdee includes workout burn
weeklyWkBurn, dailyWkBurn, deficit	reuses existing deficit	deficitWeeks, maintWeeks, cycleLabel	chosen preset, e.g. "2:1"
deficitCalories, maintCalories	both phases precomputed	proteinPct/carbsPct/fatPct	⚠ see bug note below
deficitProteinG/CarbsG/FatG	grams for cut phase	maintProteinG/CarbsG/FatG	grams for maint phase
deficitWeeksNeeded, totalCalendarWeeks	real calendar timeline	targetEndDate	—
originalGoalWeight/TargetWeeks/TargetEndDate/DailyDeficit/ActivitySettings	snapshot for exit		

Field collision: the object literal spreads `...deficitMacros` then `...maintMacros` — both use the keys `proteinPct/carbsPct/fatPct`, so the later spread silently overwrites the former. The stored top-level percentages actually end up holding the **maintenance** split (25/50/25), even during the deficit phase. The per-gram fields (`deficitProteinG` etc.) are unaffected since they're computed and stored separately before the spread.

↳ thermicEngine — *only once TE mode has been entered*

active, startedAt, currentWeight	—	bmr, sedentaryTDEE	—
dailyWorkoutBurn, fullTDEE	MET-based, weight-adjusted	dailyDeficit, foodDeficit	original deficit reused
dailyCalories	final target shown	proteinPct/carbsPct/fatPct	fixed 30/40/30 — maximize TEF
proteinG/carbsG/fatG	—	estimatedDailyRefund	TEF kcal from target macros
originalGoalWeight/TargetWeeks/TargetEndDate	snapshot for exit	originalDailyDeficit, originalStartWeight	snapshot for exit
originalActivitySettings	restored on exit		

Field Reference — logs · nbTraining · weightHistory

These three are true multi-document collections — a new document is added every time, never overwritten. Notice each one has a slightly different field set depending on *which screen* wrote it — a live example of NoSQL's schema flexibility.

M logs / {dateKey} / meals / {autoId}

1 PER LOGGED MEAL

foodName	from database or AI result	**fat**	grams
calories	—	**grams**	portion size (0 for AI entries)
protein	grams	**mealType**	breakfast / lunch / dinner / snack
carbs	grams	**loggedAt**	ISO timestamp

source OPTIONAL — only present when the meal came from the Claude-powered natural-language analyzer, set to `'ai-describe'`. A manually-picked food from the database simply omits this field entirely — Firestore documents in the same collection don't need matching shapes.

N nbTraining / {dateKey}

1 PER DAY

dateKey	YYYY-MM-DD, matches doc id	**exceededTarget**	the class label
isWeekend	Sat/Sun	**source**	'user_explicit' / 'meal_log_sum'
isDeficitPhase	false during LM/Injury/maint week	**recordedAt**	ISO timestamp
isHighStress	LM or Injury active		

W weightHistory / {autoId}

1 PER CHECK-IN

weight	kg, always present	**deficit, calories**	WeeklyCheckIn writes only OPTIONAL
loggedAt	ISO timestamp, always present	**note**	InjuryScreen writes only OPTIONAL

Two different screens write into the same collection with two different shapes: `WeeklyCheckIn.jsx` adds `{weight, loggedAt, deficit, calories}`, while `InjuryScreen.jsx` adds `{weight, loggedAt, note: 'Post-injury recovery weight'}` — no schema violation, because Firestore doesn't enforce one.

Firestore — NoSQL document database (Firebase / Google Cloud) · every field list above traced directly from the app's source, not inferred